Exps 6–8 · Multiplexers, demultiplexers, decoders

Manual pp. 19–24 and 2–3 (second numbering) · ICs 7410, 7420, 7411, 7404, 7408, 74151, 74153, 74157, 74138 · MUX data inputs checked over all 16 minterms

5 Comparator & parity → **6 MUX** → 7 DEMUX → 8 Decoder → 9–10 Flip-flops

MANUAL EXP 6 (II)

Implement f = Σm(0, 3, 5, 7, 8, 10, 14) using (i) an 8:1 MUX, (ii) a 4:1 MUX and additional gates. Viva form: "With 4 variables and an 8:1 MUX, what goes on the data inputs?"

GUESS FIRST

Three variables go to the select lines. What can each data input be, in terms of the fourth variable?

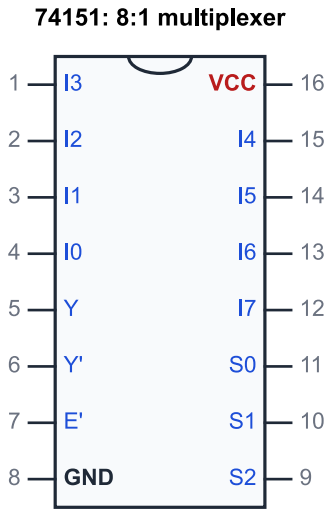
Exp 6 · Multiplexer

4:1 MUX: Y = S1'S0'D0 + S1'S0D1 + S1S0'D2 + S1S0D3.

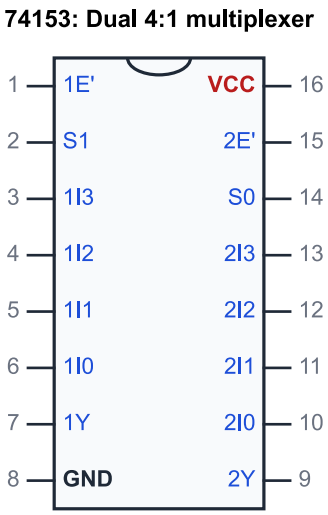
NAND-only version (Fig 6.2):

- Two 3-input NANDs with tied inputs make S1' and S0'.
- Four 3-input NANDs (D_i, select terms) feed one 4-input NAND (7420).
- That is two 7410s and one 7420.

Gate	Wires
7410 #1	S1 → 1, 2, 13 → **12 = S1'**; S0 → 3, 4, 5 → **6 = S0'**; D0 → 9, S1' → 10, S0' → 11 → 8
7410 #2	D1 → 1, S1' → 2, S0 → 13 → 12; D2 → 3, S1 → 4, S0' → 5 → 6; D3 → 9, S1 → 10, S0 → 11 → 8
7420	#1-8 → 1, #2-12 → 2, #2-6 → 4, #2-8 → 5 → **6 = Y** (pin 3 is NC)



74151 (8:1). E' (pin 7) must be LOW to enable. Y (pin 5), Y' (pin 6).



74153 (dual 4:1). S0 = pin 14, S1 = pin 2 are shared; each half has its own E'.

f = Σm(0,3,5,7,8,10,14) on the 74151 (select = A B C, data in terms of D):

Select ABC	000	001	010	011	100	101	110	111
minterms	0, 1	2, 3	4, 5	6, 7	8, 9	10, 11	12, 13	14, 15
f values	1, 0	0, 1	0, 1	0, 1	1, 0	1, 0	0, 0	1, 0
data input	**I0 = D'**	**I1 = D**	**I2 = D**	**I3 = D**	**I4 = D'**	**I5 = D'**	**I6 = 0**	**I7 = D'**
74151 pin	4	3	2	1	15	14	13	12

Wiring:

- A → pin 9 (S2), B → 10 (S1), C → 11 (S0).
- D → pins 3, 2, 1. D' (from 7404: D → 1 → 2) → pins 4, 15, 14, 12.
- Pin 13 → GND. E' (7) → GND.
- **f = pin 5.**

On the 74153 (select = A B, data in terms of C, D): I0 = (C ⊕ D)', I1 = D, I2 = D', I3 = CD'.

- S1 (pin 2) ← A, S0 (14) ← B.
- 1I0 (6) ← 7486(C, D) through a 7404 inverter.
- 1I1 (5) ← D; 1I2 (4) ← D'; 1I3 (3) ← 7408(C, D').
- 1E' (1) → GND. **f = 1Y (pin 7).**



74157 (quad 2:1). One select S (pin 1), common enable E' (pin 15). Test: S = 0 → each Y = A input; S = 1 → each Y = B input.

Exp 7 · Demultiplexer

1:4 demux with basic gates (Fig 7.1): Y0 = D S1'S0', Y1 = D S1'S0, Y2 = D S1S0', Y3 = D S1S0.

- 7404: S1 → 1 → 2 (S1'); S0 → 3 → 4 (S0').
- 7411 #1: D → 1, S1' → 2, S0' → 13 → **12 = Y0**; D → 3, S1' → 4, S0 → 5 → **6 = Y1**; D → 9, S1 → 10, S0' → 11 → **8 = Y2**.
- 7411 #2: D → 1, S1 → 2, S0 → 13 → **12 = Y3**.



74138. Address A0–A2 (pins 1–3), enables E1', E2' (active LOW) and E3 (active HIGH), outputs O0'–O7' (active LOW).

1:8 demux on the 74138 (the manual's parts list wrongly says 74151/74153):

- Select → A0 (pin 1, LSB), A1 (2), A2 (3).
- **Data D → E1' (pin 4)**; E2' (5) → GND; E3 (6) → +5 V.
- When D = 0 the chip is enabled, so the selected output goes LOW = D. When D = 1 it is disabled, so all outputs are HIGH, and the selected one again equals D.
- So **the selected output follows D, and every other output stays 1.** Outputs O0'…O7' are pins 15, 14, 13, 12, 11, 10, 9, 7.

Exp 8 · Decoder

2:4 decoder with basic gates (Fig 8.1): D0 = A'B', D1 = A'B, D2 = AB', D3 = AB.

- 7404: A → 1 → 2, B → 3 → 4.
- 7408: 7404-2, 7404-4 → 1, 2 → **3 = D0**; 7404-2, B → 4, 5 → **6 = D1**; A, 7404-4 → 9, 10 → **8 = D2**; A, B → 12, 13 → **11 = D3**.

2:4 decoder with enable, NAND only (Fig 8.2): outputs are **active-LOW**: $D_i' = (E \cdot \text{minterm}_i)'$.

- 7410 #1: A → 1, 2, 13 → 12 (A'); B → 3, 4, 5 → 6 (B'); E → 9, A' → 10, B' → 11 → **8 = D0'**.
- 7410 #2: E, A', B → 1, 2, 13 → **12 = D1'**; E, A, B' → 3, 4, 5 → **6 = D2'**; E, A, B → 9, 10, 11 → **8 = D3'**.

E A B	0 x x	1 0 0	1 0 1	1 1 0	1 1 1
D0' D1' D2' D3'	1 1 1 1	**0** 1 1 1	1 **0** 1 1	1 1 **0** 1	1 1 1 **0**

TRAP: ACTIVE-LOW OUTPUTS

On the trainer, the selected output's LED goes **out** while the others stay lit, for both the NAND decoder and the 74138. That's correct, not a fault.

WRITE IN YOUR RECORD

Result: the 4:1 MUX (NAND gates) and the 74151/74153/74157 function tables were verified. f = Σm(0, 3, 5, 7, 8, 10, 14) was realised on the 74151 (I0 = D', I1 = I2 = I3 = D, I4 = I5 = I7 = D', I6 = 0) and on the 74153 with gates. The 1:4 DEMUX (gates), the 1:8 DEMUX (74138, data on E1'), and the 2:4 decoders (basic gates; NAND with enable, active-low) were verified.

Theory for the record: a multiplexer selects one of 2^n data inputs using n select lines: $Y = \Sigma\, m_i D_i$. Any function of n + 1 variables can be built on a 2^n:1 MUX by putting n variables on the selects and 0, 1, x or x' on each data input. A demultiplexer routes one input to one of 2^n outputs. A decoder activates exactly one of 2^n outputs for an n-bit input. A decoder with an enable becomes a demultiplexer when the data is applied to the enable.

CHECK 1

(a) Full adder on one 8:1 MUX (select A, B, C_{in}): list the data inputs for S and for C_{out}. (b) The 74151's E' is left floating. What do you see? (c) Why does a decoder plus an OR gate realise any function?

Answers

Opening question. Each data input is **0, 1, D or D'**. Read each pair of minterms that share the same select value (see the table).

Check 1. (a) S: I1, I2, I4, I7 = 1, others 0. C_{out}: I3, I5, I6, I7 = 1, others 0 (two 74151s, or one 74153 per output with S = C_{in}, C_{in}', C_{in}', C_{in} and C_{out} = 0, C_{in}, C_{in}, 1). (b) Floating acts HIGH, which disables the chip: Y stays 0 and Y' stays 1 whatever the inputs. (c) The decoder produces every minterm. OR-ing the minterms in Σm gives the function. (With active-low outputs, use a NAND instead of the OR.)